

Projeto de Predição de Arremessos do Kobe Bryant

Projeto desenvolvido como parte da disciplina Engenharia de Machine Learning na pós-graduação em Inteligência Artificial do Instituto Infnet.

O objetivo principal foi construir um preditor para classificar se um arremesso realizado por Kobe Bryant resultou em cesta (acerto) ou não (erro), utilizando duas abordagens de classificação: Regressão Logística e Árvore de Decisão.

Tecnologias Utilizadas

- Kedro – para estruturação dos pipelines e modularização do projeto
- PyCaret – para automação de modelagem
- MLflow – para rastreamento de experimentos e versionamento de modelos
- Scikit-learn – como backend de modelos
- Pandas / NumPy / Matplotlib – para manipulação de dados
- Streamlit – para criação de dashboards interativos

Como Rodar o Projeto

1. Clone o repositório:

```
git clone <url_do_seu_repositorio>
cd kobe
```

2. Instale as dependências:

```
pip install -r requirements.txt
```

3. Execute os pipelines na ordem correta:

```
kedro run --pipeline preparacao_dados
kedro run --pipeline treinamento
kedro run --pipeline aplicacao
```

4. Execute o dashboard:

```
streamlit run dashboard.py
```

Respostas às Questões Teóricas do Projeto

1. Como as ferramentas Streamlit, MLFlow, PyCaret e Scikit-learn auxiliam na construção dos pipelines descritos anteriormente?

- a. Rastreamento de Experimentos: O MLflow foi utilizado para registrar os experimentos com métricas (log loss e F1 para este projeto), parâmetros e versões dos modelos.
- b. Funções de Treinamento: PyCaret foi usado para facilitar a configuração e treinar os modelos (regressão logística e árvore). Scikit-learn foi usado para métricas personalizadas e o Pycaret automatiza tarefas como pré-processamento, validação e finalização dos modelos.
- c. Monitoramento da Saúde do Modelo: O Streamlit permite acompanhar os resultados da aplicação do modelo em tempo real e o MLflow permite monitoramento via logs de métricas.
- d. Atualização de Modelo: O MLflow gerencia versões de modelos permitindo rastreabilidade.
- e. Provisionamento (Deployment): A aplicação final do modelo é feita no pipeline aplicacao e visualizada no dashboard. O modelo é carregado diretamente via URI do MLflow.

2. Com base no diagrama realizado, aponte os artefatos que serão criados ao longo do projeto.

**** 2.1. Dados de Entrada****

- `dataset_kobe_dev.parquet` : Dados históricos utilizados para treino e validação.

- `dataset_kobe_prod.parquet` : Dados recentes usados para simular a produção.

**** 2.2. Dados Processados ****

- `data_filtered` : Resultado da filtragem dos dados (remoção de nulos + seleção de colunas).
- `base_train` : 80% do `data_filtered` usados para treinar os modelos.
- `base_test` : 20% do `data_filtered` usados para avaliação dos modelos.

**** 2.3. Modelos Treinados ****

- `modelo_reg` : Modelo de regressão logística treinado com PyCaret.
- `modelo_arvore` : Modelo de árvore de decisão treinado com PyCaret.

**** 2.4. Métricas de Avaliação (Validação) ****

- `log_loss_regressao` : Log loss da regressão logística.
- `log_loss_arvore` : Log loss da árvore de decisão.
- `f1_score_arvore` : F1 Score da árvore de decisão.

Observação: O projeto exige apenas o cálculo de F1 para a árvore.

**** 2.5. Resultados da Aplicação ****

- `resultados_aplicacao.parquet` :
 - Predições feitas sobre a base de produção.
 - Contém colunas:
 - `prediction` : acerto/erro previsto.
 - `probability` : probabilidade de acerto estimada pelo modelo.
- `metrics_aplicacao.json` :
 - Métricas do modelo aplicado em produção:
 - `log_loss_aplicacao`
 - `f1_score_aplicacao`

**** 2.6. Visualização****

- `dashboard.py` : Interface Streamlit que:
 - Lê os resultados e métricas da aplicação
 - Exibe os valores no painel
 - Plota gráfico com arremessos (acertos em azul, erros em vermelho)

3. Selecione um dos dois modelos para finalização e justifique sua escolha.

O modelo selecionado foi árvore de decisão.

Esse modelo foi avaliado com log loss e F1 score e teve um desempenho satisfatório ($f1_score=0.643$), isso indica equilíbrio entre precisão e recall.

Como os requisitos do projeto pedem F1 score apenas para árvore, não foi possível comparar essa métrica para os dois modelos.

Dessa forma, foi utilizado log loss como base de decisão.

A função `escolher_modelo` do node treinamento retorna o modelo com menor log loss.

4. O modelo é aderente à nova base? O que mudou entre uma base e outra? Justifique.

Sim. As métricas da aplicação ($\log_loss \approx 0.676$, $f1_score \approx 0.643$) mostram comportamento semelhante ao observado durante o treino.

5. Descreva como podemos monitorar a saúde do modelo no cenário com e sem a disponibilidade da variável resposta para o modelo em operação.

Para monitorar a saúde do modelo com variável resposta, podemos comparar prediction com `shot_made_flag` e registrar métricas como acurácia e f1.

Para fazer o mesmo sem variável resposta: monitorar distribuição de probabilidades, proporção de classes previstas e realizar detecção de drift estatístico nos dados de entrada.

6. Descreva as estratégias reativa e preditiva de retreinamento para o modelo em operação.

Estratégia reativa: retreinar quando houver queda no desempenho ou detecção de drift.

Estratégia preditiva: retreinar periodicamente ou com base em volume acumulado de novos dados.

8. Link para o Repositório

(Inserir o link real do GitHub/Bitbucket/GitLab aqui antes da entrega final)